

From Classical ML to Neural Networks

A neural network extends classical ML: **linear model + nonlinear representation learning + gradient-based learning.**

- Starting point: logistic regression
- Destination: ReLU MLP & Gradient Descent
- Core proof: a deep network without activation collapses into a single affine map.
- Core principle: Optimization is walking down the Loss Surface; Backpropagation is applying the chain rule repeatedly.

Where Week 1 Fits

Week 1 builds the foundational grammar needed for RNNs, Attention, and Transformers.

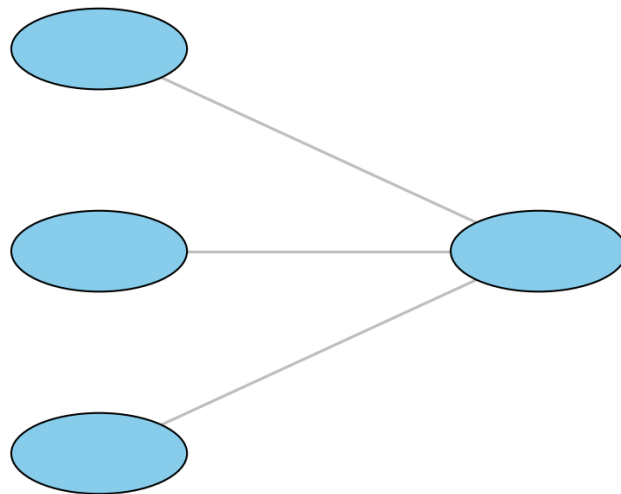
- Week 1: Affine maps, MLPs, Activations, Optimization, Backprop
- Week 2: RNN, LSTM, GRU
- Week 3: Attention Mechanics
- Week 4: Transformer Architecture
- Week 5: Paper Reading & Wrap-up

Logistic Regression as a Single Layer

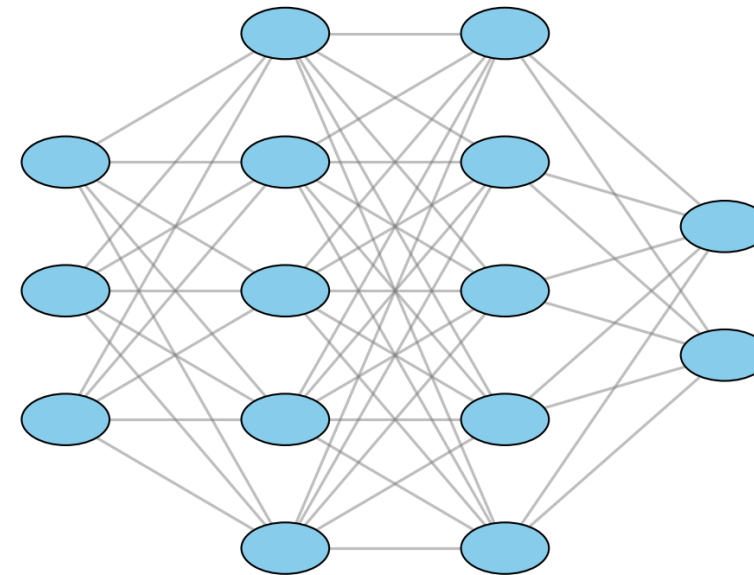
Binary logistic regression is the simplest Neural Network, composed of one affine layer and a sigmoid.

Network Structure

Single Layer (Logistic Regression)



Multi-Layer Perceptron (MLP)



- A Single Layer model divides the input space with a single flat hyperplane.
- A Multi-Layer Perceptron (MLP) learns hidden representations to create nonlinear decision boundaries.

Start with Tensor Shapes

Deep learning equations become much clearer when you track tensor shapes.

Shapes

$$\begin{aligned} X &: [\text{batch}, d_{in}] \\ W &: [d_{in}, d_{out}], \quad b : [d_{out}] \\ Y &= XW + b : [\text{batch}, d_{out}] \end{aligned}$$

When Linear Boundaries Fail

Model failures are often due to a lack of 'Representation' power, not just poor 'Optimization'.

- Nonlinear datasets like XOR or `make_moons` cannot be separated by a simple line.
- No single straight line can divide the two classes. No matter how well you optimize, the model hits a structural limit.

Deep Linear Collapse

Stacking multiple linear layers does not escape the limits of logistic regression.

Derivation

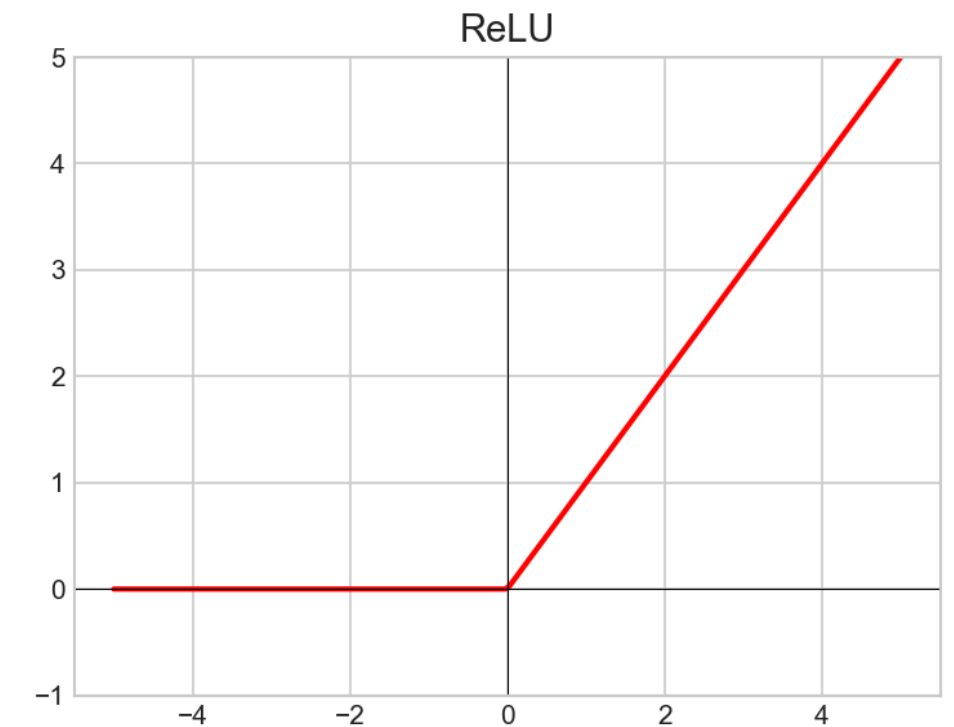
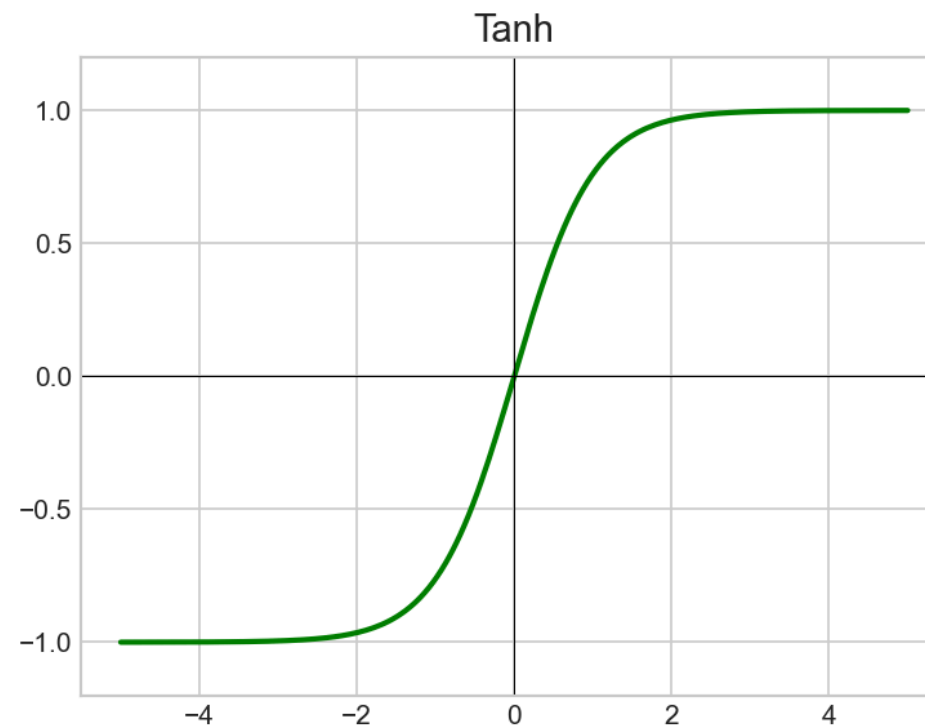
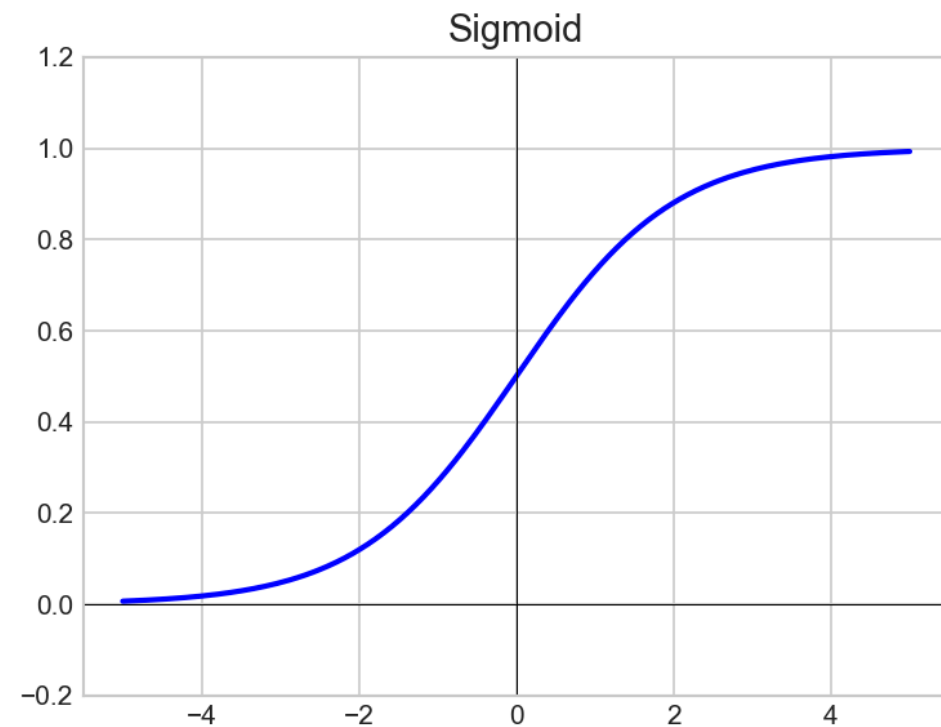
$$\begin{aligned}H &= XW_1 + b_1 \\Z &= HW_2 + b_2 \\Z &= (XW_1 + b_1)W_2 + b_2 \\&= X(W_1W_2) + (b_1W_2 + b_2) \\&= XW_* + b_*\end{aligned}$$

- Without activation functions, stacking multiple layers mathematically collapses back into a single affine map.

Activation Functions

Activations inject nonlinearity between layers, preventing collapse and allowing boundaries to bend.

Activations



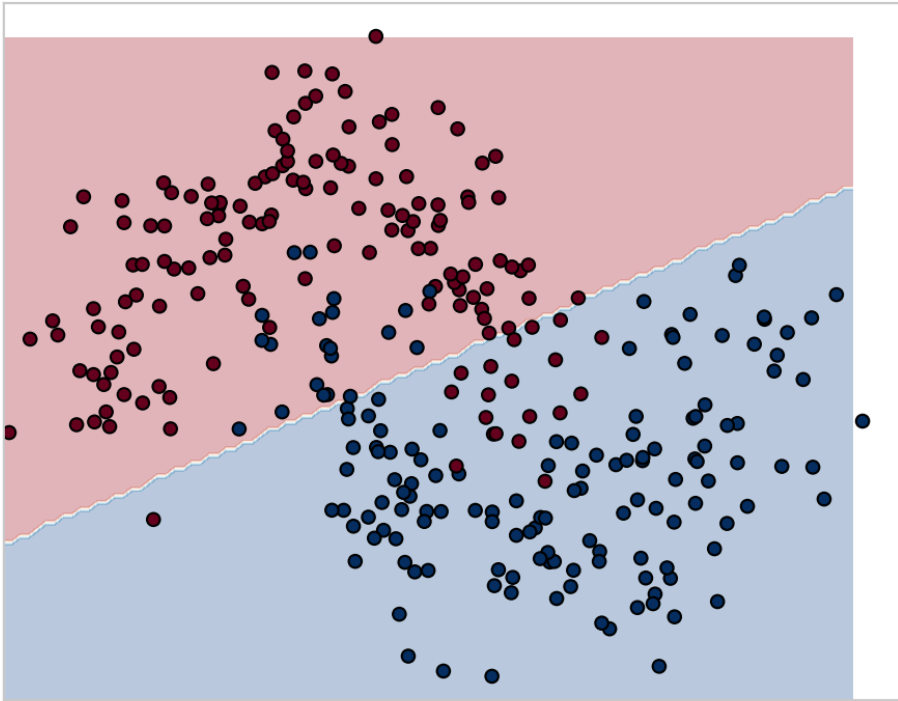
- Sigmoid: Outputs $[0, 1]$. Good for probabilities but suffers from vanishing gradients in hidden layers.
- Tanh: Outputs $[-1, 1]$, zero-centered.
- ReLU: $\max(0, x)$. Discards negative values and passes positive gradients perfectly. The standard choice today.

Demo: Decision Boundaries

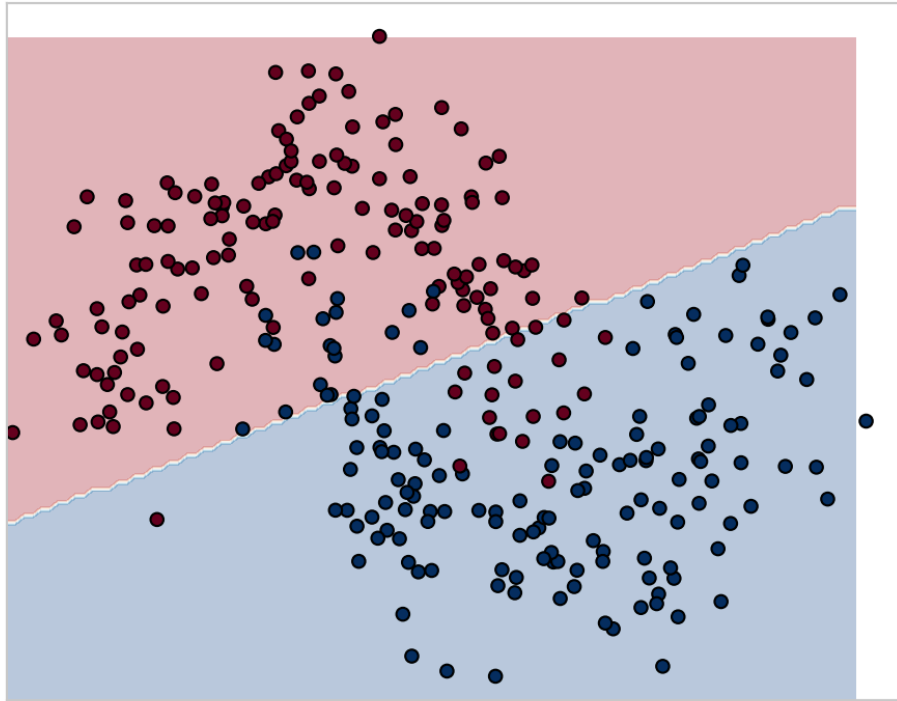
A ReLU MLP can learn nonlinear boundaries through its hidden representations.

Decision Boundaries

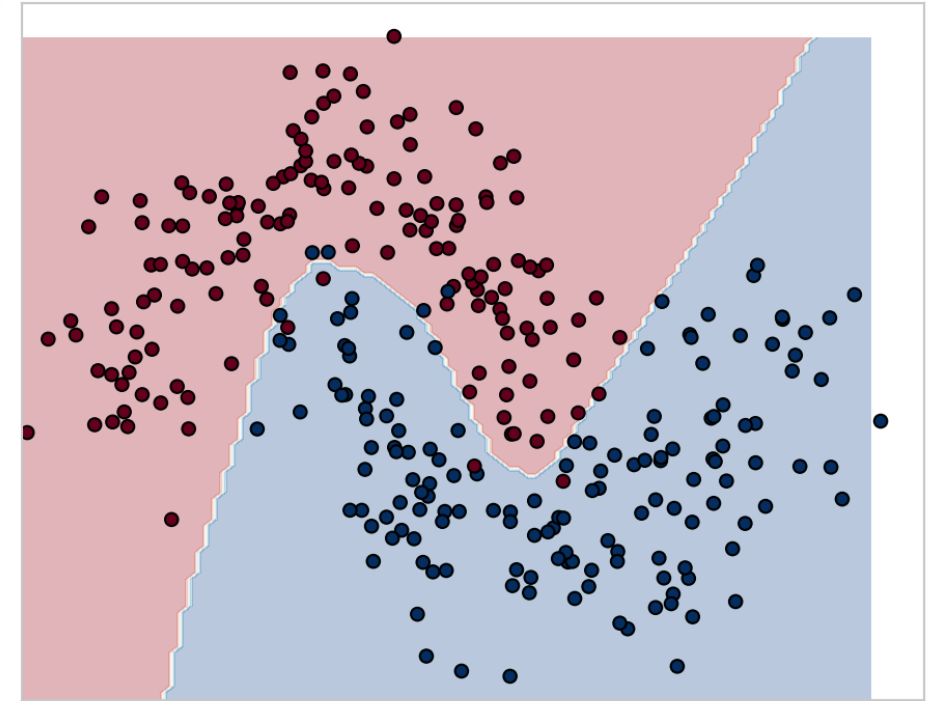
Logistic Regression (Linear)



MLP w/o Activation (Linear Collapse)



MLP with ReLU (Non-linear)



- Logistic Regression: Fails with a single straight line.
- MLP w/o Activation: Complex structure but still a linear boundary (proving Collapse).
- MLP with ReLU: Perfectly separates the data by bending the space nonlinearly.

Output Layer and Cross Entropy Loss

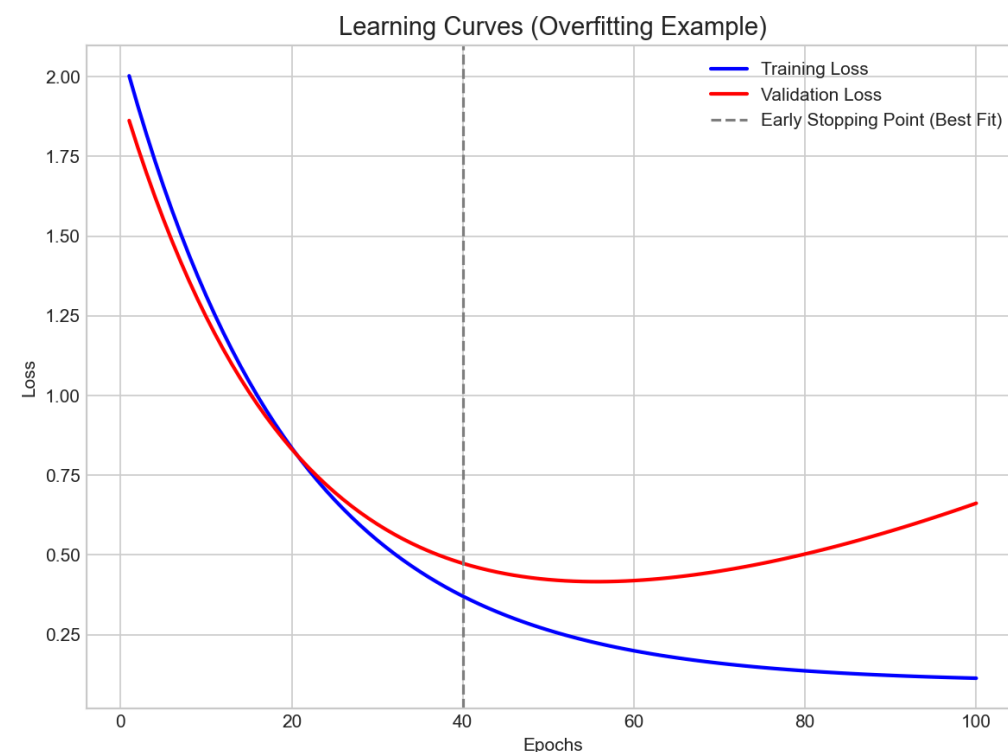
The Loss is a learning signal that drives the model to assign high probability to the true class.

- Logits -> Softmax -> Probabilities
- Cross Entropy Loss: $-\log(p_y)$ (where p_y is the predicted probability for the true class y).
- As the probability approaches 1, Loss approaches 0. Incorrect predictions cause the loss to explode exponentially, applying a strong penalty.

Loss vs Metric, and Learning Curves

Loss is a differentiable objective for parameter updates; Metric is a human-readable performance standard.

Loss Curves

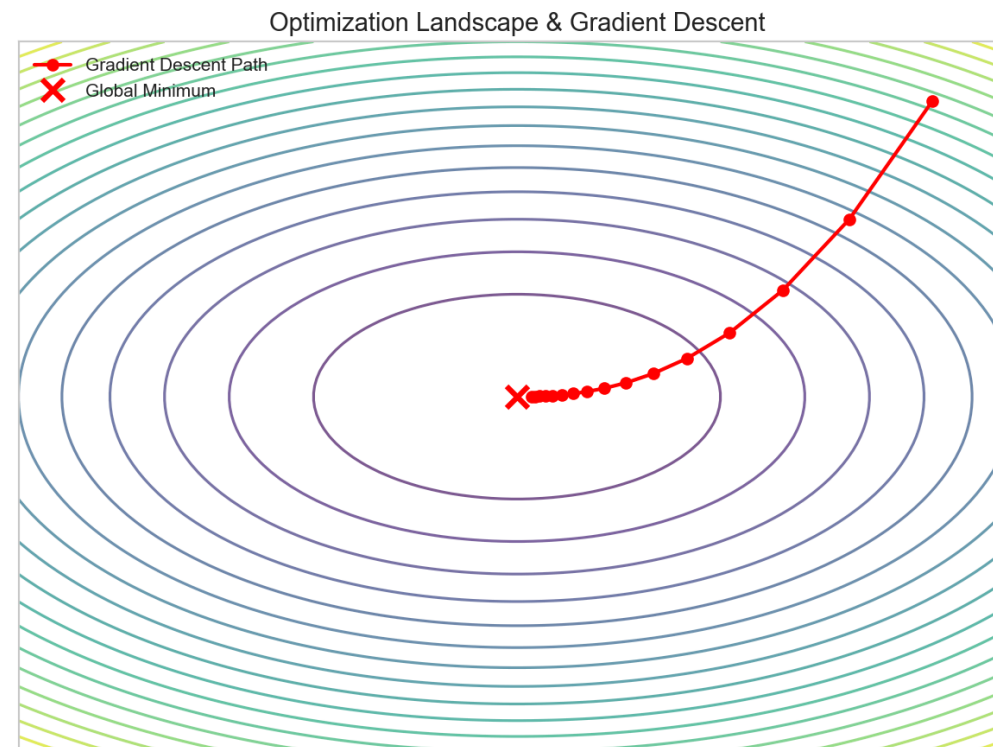


- Training Loss: How well the model fits the training data.
- Validation Loss: Evaluated on unseen data. When it starts rising, the model is memorizing noise (Overfitting).
- Proper training includes Early Stopping at the optimal point.

Optimization Landscape & Gradient Descent

Parameter optimization is like walking down a foggy mountain (Loss Surface) to find the lowest valley (Minimum).

Gradient Descent



- Gradient: The direction of steepest ascent at the current position.
- Parameters move in the 'opposite' direction of the gradient to reduce the Loss.
- Learning Rate (Step Size): If too small, convergence is slow. If too large, it overshoots and diverges.

Batch vs Stochastic Gradient Descent (SGD)

Computing the gradient over the entire dataset (Full Batch) is too slow. Mini-batches offer a noisy but faster path.

- Full Batch GD: Accurate direction, but only 1 parameter update per epoch.
- SGD (Mini-batch): Uses a subset of data. Moves in a noisy zigzag path but updates frequently within 1 epoch.
- This 'noise' actually helps the model escape Local Minima and Saddle Points.

Backpropagation Basics

Solving derivatives for millions of parameters analytically is impossible. We traverse the Computational Graph backward.

- Forward Pass: Input **X** passes through the model to produce **Y_hat** and **Loss**. (Intermediate values are cached).
- Backward Pass: Starting from the **Loss**, gradients are passed backward from Output -> Hidden -> Input layers.
- The 'Error signal' (how wrong we were) flows backward, assigning responsibility to each parameter.

Chain Rule Intuition

$\frac{dz}{dx} = \frac{dz}{dy} * \frac{dy}{dx}$. Complex derivatives are just multiplications of local gradients.

- Each Layer only knows how to compute its 'Local Gradient' (derivative of its output w.r.t its input).
- It multiplies the incoming gradient from the layer ahead by its own local gradient, passing the result backward.
- Calling `loss.backward()` in PyTorch automatically executes this chain rule under the hood.

PyTorch Minimal Training Loop

The Forward, Loss, Backward, and Update sequence maps perfectly to 4 lines of code.

PyTorch Code

```
logits = model(X_batch)           # 1. Forward Pass
loss = criterion(logits, y_batch)  # 2. Compute Loss

optimizer.zero_grad()             # 3. Clear old gradients!
loss.backward()                   # 4. Backpropagation (computes grad for all parameters)
optimizer.step()                  # 5. Gradient Descent (updates W to W_new)
```

Connecting to Paper-Reading Language

Today's concepts form the framework for understanding experimental setups in research papers.

Terminology

Research Term	Classroom Meaning
Optimization Landscape	The high/low terrain of Loss values across parameter combinations
Local Minima / Saddle Point	Points where the gradient is zero, stalling training, but not the global minimum
Overfitting / Early Stopping	Stopping training when Validation Loss starts to increase
Ablation Study	An experiment that removes a component (e.g., Activation) to prove its impact

Assignment & End-of-class Checkpoint

Review these questions and check your assignment.

- 1. What is the structural difference between a Single Layer and an MLP, and how does it affect the decision boundary?
- 2. Mathematically, why does stacking multiple layers with 'identity' activations fail on the `make_moons` dataset?
- 3. What happens during Gradient Descent if the Learning Rate is set too high?
- 4. What mathematical rule is implicitly applied when you call `loss.backward()` in PyTorch?
- Assignment: Write the code for the `make_moons` classification experiment (Visualize Logistic Regression vs MLP).
- Extra Experiment: Set the Learning Rate to an extreme high (10.0) and extreme low (1e-4). Observe how the loss curve breaks and explain why in one sentence.

